



Canvas vs Blackboard vs Moodle vs D2L Brightspace API limits, OAuth scopes, and LTI 1.3 endpoints compared in one interactive dashboard

Computer

Build a developer migration guide for Canvas, Blackboard, Moodle, and D2L Brightspace REST and LTI 1.3 service integrations

Computer

Show me the full comparison table

How do LTI 1.3 endpoints differ across them

What OAuth scopes does Canvas require also anything else to make it feel like one system consisten throughout and not a bunch of diffrent screens and tools

To make Semester feel like **one coherent system**, the integration architecture matters—but so do consistent language, navigation, design tokens, permissions, source labels, recovery behavior, and a shared object model. The LMS integrations should feel like connected sources inside Semester, not separate mini-products embedded in different screens.

Canvas is the clearest published example for LTI Advantage scopes: it requires a tenant-issued LTI Developer Key, an RSA-signed OAuth 2.0 client-credentials token flow for Advantage services, and explicit service scopes such as NRPS roster read and AGS line-item/score/result permissions. ^[1658]_[1659]^[^1665]

Full LMS comparison

Dimension	Canvas	Blackboard Learn	Moodle	D2L Brightspace	Semester standard
Primary integration baseline	LTI 1.3/LTI Advantage plus REST APIs	LTI 1.3/LTI Advantage plus REST APIs	LTI 1.3, Web Services, plugins, custom APIs	LTI 1.3/LTI Advantage plus Brightspace Developer Platform	LTI 1.3/LTI Advantage first; proprietary APIs only when needed
LTI registration	Developer Key + tool configuration; static or hosted JSON supported	Admin registration/approval and placement configuration	Site-admin configuration; behavior depends on version/plugins	Tenant-admin registration/configuration	Secure per-tenant registration record and onboarding checklist
OIDC login initiation	Platform supplies per-registration/login configuration	Platform supplies tenant registration values	Moodle instance supplies values	Brightspace tenant supplies values	Validate state, nonce, iss, aud, exp, deployment ID, redirect URI
JWKS/key model	Tool public JWK statically configured or dynamically served by a URL Canvas can access	Platform/tool JWKS configuration varies by deployment	Instance/tool configuration varies	Tenant/tool configuration varies	Support rotation; pin issuer/domain; log kid; graceful renewal/failure handling

Dimension	Canvas	Blackboard Learn	Moodle	D2L Brightspace	Semester standard
NRPS	Supported with explicit contextmembership.readonly scope	Validate per environment/admin configuration	Version/configuration dependent	Brightspace supports LTI Advantage Complete, including NRPS v2	Request minimum roster/role fields; do not assume roster service
AGS	Supported with enabled AGS scopes for line items, scores, and results	Requires explicit access/configuration; validate by deployment	Version/configuration dependent	Brightspace supports AGS v2 in LTI Advantage Complete	Grade writes are explicit, idempotent, previewable, audited, and reconciled
Deep Linking	Supported through configured capability	Admin/tool placement dependent	Version/configuration dependent	Brightspace supports Deep Linking v2	Optional; always offer manual content workflow
REST/API auth	Developer Key plus OAuth/API token model	OAuth 2 application registration; verify tenant endpoint behavior	Web-service tokens and admin-enabled services; OAuth 2 exists for external-service configuration	Brightspace app/API credentials and REST-like APIs	OAuth 2/OIDC, scoped credentials, secret rotation, tenant isolation
Rate limits	Dynamic request-cost quota; 429 throttling	Verify by deployment; no universal published production limit	Host/configuration dependent; no universal limit	Token-bucket credits; response headers and 429 behavior	Adaptive queue, backoff/jitter, circuit breaking, durable reconciliation
Events/webhooks	Validate platform event capabilities per tenant	Validate API/event capability per Learn deployment	Plugins/custom hosting may determine behavior	Validate available event/subscription mechanisms per tenant	At-least-once internal event model, signed outbound webhooks, reconciliation
Biggest integration risk	Incorrect Developer Key/scopes and differing tenant permissions	Hosting/version/admin configuration variance	Plugin, version, hosting, and security-governance variance	Tenant entitlement/configuration and variable request cost	Never infer capability from vendor name alone—test in customer sandbox
Best Semester approach	Use LTI first, Canvas API only for approved gaps	Capability-discovery first, then LTI/API as approved	Favor LTI/Web Services; avoid dependency on custom plugins	Use LTI first, Developer Platform APIs for explicit approved flows	Publish supported capability tiers and per-LMS integration test results

Brightspace’s developer platform states that it supports LTI Advantage Complete—including LTI 1.3 Core, AGS v2, Deep Linking v2, and NRPS v2—while Moodle’s developer documentation exposes both LTI/platform capabilities and administrative OAuth 2 configuration. ^[1666][1643][^1645]

LTI endpoint differences

The LTI 1.3 protocol uses the same types of endpoints across platforms, but the **actual URLs, enabled services, claim release, registration process, and permissions differ per tenant**. Do not hardcode vendor endpoint patterns.

Common endpoint record

For each Semester customer, store a separate encrypted LTI registration object:

```
tenant_id
lms_provider
lms_version
environment: sandbox / production
issuer
client_id
deployment_id
oidc_login_initiation_url
authorization_url
jwks_url
access_token_url
redirect_uris
deep_linking_endpoint
nrps_context_memberships_url
ags_lineitems_url
ags_results_url
ags_scores_url
enabled_scopes
enabled_message_types
claims_released
key_rotation_method
admin_owner
last_validated_at
fallback_mode
```

How they differ in practice

Endpoint/control	Canvas	Blackboard	Moodle	D2L Brightspace	Semester handling
Issuer (iss)	Canvas tenant/domain-specific	Learn deployment-specific	Moodle site-specific	Brightspace tenant-specific	Exact-match allowlist by tenant —not brand-level matching
OIDC login	Defined by Canvas tool registration/configuration	Created as part of Learn tool setup	Defined by instance/site setup	Defined by tenant tool setup	Validate state, redirect URI, and login context
Authorization/JWKS	Canvas-specific domain endpoints; tool config/key relationship	Learn deployment-specific URLs	Moodle site endpoints	Brightspace tenant endpoints	Discover/register from admin-provided configuration; cache JWKS by issuer and kid
Token endpoint	Canvas LTI Advantage client-credentials flow	Depends on Learn deployment/services	Depends on LTI service configuration	Depends on Brightspace tenant/service configuration	Treat as tenant configuration; no hardcoded URL templates
NRPS endpoint	Provided in LTI launch claim only when enabled and scoped	May be present when enabled	May be present when configured	Provided when NRPS is enabled	Read endpoint from verified launch/service claim; avoid using guessed REST endpoint
AGS endpoints	Provided as AGS endpoint claim when enabled	Tenant/admin-configured service availability	Version/configuration dependent	Provided when AGS enabled	Use claim-provided line-items endpoint; retain per-resource context and scope

Endpoint/control	Canvas	Blackboard	Moodle	D2L Brightspace	Semester handling
Deep Linking endpoint	Supplied through deep-link request flow/configuration	Placement/configuration specific	Configuration/version specific	Tenant-configured	Feature-detect; provide manual fallback
Key rotation	Canvas can fetch public JWKs from tool-hosted URL	Varies	Varies	Varies	Support static/dynamic keys, expiration alerts, dual-key overlap, and rotation runbook

LTI launches must be validated from the signed claims, not from expected URL shape. Canvas specifically notes that a provider serving multiple institutions should store and look up the correct Developer Key based on launch parameters such as the Canvas API domain; its LTI Advantage services use the OAuth 2.0 client-credentials grant. [^1665]

Canvas OAuth scopes

Canvas has two distinct authorization concepts that should not be mixed:

1. **Canvas REST API permissions/scopes** tied to Developer Key configuration and institutional authorization.
2. **LTI Advantage service scopes** used through a JWT-authenticated OAuth 2.0 client-credentials token for the service endpoints released in the launch context.

Minimum Canvas LTI Advantage scopes

Semester use case	Canvas LTI scope	Use only when needed
Read course roster and roles	https://purl.imsglobal.org/spec/lti-nrps/scope/contextmembership.readonly	Yes, if roster/role synchronization is necessary
Create/read/update/delete gradebook line items	https://purl.imsglobal.org/spec/lti-ags/scope/lineitem	Only for assessments that need Semester-managed grade columns
Read a specific line item	https://purl.imsglobal.org/spec/lti-ags/scope/lineitem.readonly	Prefer this where write access is not needed
Post scores	https://purl.imsglobal.org/spec/lti-ags/scope/score	Only if authorized human/institution workflow writes grades
Read results	https://purl.imsglobal.org/spec/lti-ags/scope/result.readonly	Only if Semester must reconcile released grades/results
Read Canvas page content through postMessage	https://canvas.instructure.com/lti/page_content/show	Only for the explicit page-content feature

Canvas documents that NRPS requires the `contextmembership.readonly` scope and that AGS requires enabled Developer-Key scopes; it supports Line Items, Score, and Result services, and a tool must be available in the course plus complete OAuth 2 client credentials before making AGS requests. [^1658][^1659][^1661][^1664]

Canvas permission model

Institutional administrators also control who can configure/manage LTI tools and roles. Canvas exposes permissions such as `manage_lti_add`, `manage_developer_keys`, and `manage_grades`; Semester should never assume an instructor can install or authorize an enterprise integration. [^1660][^1667]

Secure token flow

```
Semester receives LTI 1.3 launch
→ validates JWT signature and core claims
→ identifies tenant and deployment
→ reads allowed service endpoints and scopes from verified claims
→ signs a client-credentials token request using tenant-specific RSA private key
→ requests only required scope(s)
→ receives short-lived access token
→ calls only the released endpoint(s) for that course/resource context
→ writes an audit event
→ caches token only until expiry, encrypted and tenant-scoped
→ retries safely with rate-limit controls
```

Make it feel like one system

The most important way to avoid “a bunch of different screens and tools” is to make every module obey the same **platform grammar**.

One information architecture

Use a stable top-level structure everywhere:

```
Today
Learn
Plan
Community
Opportunities
Support
Messages
Search
Me
```

Then use consistent contextual navigation:

```
Institution → Term → Course / Community / Service → Object
```

Example:

```
Vanderbilt University
→ Fall 2026
→ ECON 210
→ Assignment 2
→ Source card / rubric / submission / feedback
```

A student should never need to understand which backend system owns an item in order to use it.

One object model

Every module should use the same shared objects and semantics:

```
Person
Institution
Role
Term
Course
Community
Service
Opportunity
Task
```

Event
Document/source
Plan
Assessment
Submission
Feedback
Grade
Share
Notification
Conversation
Integration
Audit event

Every object carries:

Owner
Source
Authority level
Scope/permissions
Status/freshness
Version
Retention class
Accessibility metadata
Audit history
Recovery/undo state

This is how an assignment, appointment, club event, career opportunity, and financial-aid checklist can feel like parts of one platform even though their underlying systems differ.

One design system

Create a centralized design system and do not allow modules to invent their own interface language.

Design tokens:
Color, typography, spacing, elevation, motion, borders, focus, breakpoints.

Components:
App shell, top bar, side navigation, breadcrumb/context switcher, cards, tables, empty states, forms, date/time controls, dialogs, drawers, toasts, status chips, source cards, notifications, timeline, activity log, attachment picker, command palette.

Interaction rules:
Loading, saving, error, conflict, offline, retry, confirmation, undo, permission denied, locked, archived, and success states.

Accessibility rules:
Keyboard path, focus order, labels, validation, live regions, contrast, zoom/reflow, motion preferences, accessible tables/charts, non-drag alternative, captions/transcripts, plain-language errors.

One trust pattern: Source, Scope, Status

Use the same trust component in every product area.

Source:
Where did this come from?

Scope:
Who can see or act on it?

Status:

Is it official, estimated, draft, current, pending, stale, restricted, archived, or needs review?

Examples:

```
Assignment deadline
Source: Instructor published
Scope: ECON 210, Fall 2026
Status: Current

Transfer-plan course match
Source: Student-provided syllabus + published catalog
Scope: Private planning workspace
Status: Estimated; official review required

Basic-needs referral
Source: Student request
Scope: Shared with Basic Needs Center
Status: Awaiting intake

Grade
Source: Instructor released via Semester
Scope: Student and authorized course staff
Status: Released; external LMS sync complete
```

One action model

Every action should follow the same behavior:

```
Preview
→ explain impact
→ identify target and authority
→ confirm if consequential
→ execute
→ show completion state
→ preserve audit event
→ provide undo/recovery if possible
→ show support/escalation route if not
```

For example, adding an assignment deadline, sharing a portfolio, syncing a grade, joining a club, or sending a referral should all make scope, impact, and recovery obvious.

One notification center

A user should not receive different reminder experiences from courses, clubs, plans, support, or career tools.

```
Unified inbox and notification center
Notification purpose and source label
In-app, email, push, and optional SMS channels
Quiet hours and schedule preferences
Urgency levels
Digest controls
Duplicate suppression
Per-course/community/service controls
Delivery history
Why-am-I-seeing-this explanation
```

One search and command system

Global search:
Courses, assignments, sources, services, clubs, events,
opportunities, messages, support articles, settings.

Command palette:
Add task
Find a source
Ask Semester
Start a study session
Prepare advisor agenda
Find support
Join event
Upload work
Review feedback
Manage sharing
Report an issue

Search results must respect tenant, course, role, source, privacy, and retention boundaries—never blend private content from unrelated contexts.

One profile and data-agency center

Profile and accessibility preferences
Notification preferences
Connected accounts
Active shares
Mentorship/community privacy
AI settings/history where applicable
Data export/deletion
Support-access history
Security sessions/MFA
Consent and policy history

A student should not need to hunt through course, community, career, and support modules to understand who has access to their information.

One operational control plane

Internally, do not create separate support/admin tools for every module. The Operations Console should unify:

Tenant configuration
Entitlements
Feature flags
Integration health
Content freshness
Support cases
Incidents
AI policy
Privacy/consent
Accessibility issues
Contracts/customer commitments
Billing
Audit/evidence
Release impact

The console can use domain-specific views, but must use the same identity, permission, audit, case, notification, and runbook services.

Consistency release gates

Before releasing any new Semester feature, require:

- [] Uses the shared application shell and context switcher.
- [] Uses shared design-system components; no one-off controls without approval.
- [] Displays Source, Scope, and Status where information or authority matters.
- [] Uses standard loading, error, empty, permission, confirmation, recovery, and support patterns.
- [] Appears in global search and command palette where appropriate.
- [] Uses the unified notification system.
- [] Honors shared profile, accessibility, privacy, retention, and sharing controls.
- [] Emits standard audit/activity events.
- [] Passes keyboard, screen-reader, zoom/reflow, mobile, and low-bandwidth tests.
- [] Has one named owner, source-freshness model, support runbook, and analytics definition.
- [] Can be enabled/disabled by tenant through the same entitlement and feature-flag system.
- [] Has migration/offboarding/export behavior defined.

Final architecture principle

Build one platform with many domains, not many products stitched together:

- One identity
- One context
- One object model
- One design system
- One trust pattern
- One search system
- One notification system
- One data-agency center
- One audit/event model
- One integration gateway
- One operations console

When every new service—LMS, study tools, grading, clubs, career, basic needs, AI, or administration—must plug into those shared foundations, Semester will feel coherent even as it becomes much broader.

✱✱

1. <https://moodledev.io/docs/5.0/apis>
2. <https://developerdocs.instructure.com/services/canvas/oauth2/file.oauth>
3. <https://docs.valence.desire2learn.com/about.html>
4. <https://developerdocs.instructure.com/services/canvas/permissions/file.permissions>
5. <https://developerdocs.instructure.com/services/dap/dataset/dataset-namespaces/dataset-canvas>
6. <https://developerdocs.instructure.com/services/canvas/external-tools/lti>
7. https://developerdocs.instructure.com/services/canvas/external-tools/lti/file.tools_intro
8. https://developerdocs.instructure.com/services/canvas/external-tools/lti/file.lti_dev_key_config
9. https://developerdocs.instructure.com/services/canvas/resources/developer_keys
10. https://docs.moodle.org/en/OAuth_2_authentication
11. https://developerdocs.instructure.com/services/canvas/external-tools/lti/file.assignment_tools
12. <https://developerdocs.instructure.com/services/canvas/external-tools/lti/file.provisioning>
13. <https://developerdocs.instructure.com/services/canvas/resources/roles>
14. https://developerdocs.instructure.com/services/canvas/resources/names_and_role
15. https://developerdocs.instructure.com/services/canvas/external-tools/file.canvas_roles
16. https://developerdocs.instructure.com/services/canvas/resources/lti_registrations

17. https://developerdocs.instructure.com/services/canvas/external-tools/lti/placements/file.navigation_tools